AIQA — Backend Test Self-Contained Reliability Fix Report

Date: 2026-08-25 **Branch:** `claude/run-this-9eeb2e` **Scope:** Make `cargo test` fully reproducible in a clean environment — no manually-started backend on port 8080, no fixed replacement port, no weakened SSRF, no newly-ignored tests.

---

1. Executive Summary

Exactly 3 backend tests genuinely depended on a real, manually-started orchestrator already listening on `127.0.0.1:8080` — not 5, not an assumed count, verified by actually running the affected group with no server up and reading the real failures. All 3 live in `api_tests.rs` and share one root cause: `const HEALTH_URL: &str = "http://127.0.0.1:8080/health"` pointed at a target nothing in the test process itself ever started.

The production SSRF guard (`executor::is_blocked_ip`, unmodified) only ever allows a loopback target on the literal port `8080` — every other loopback port, including any OS-assigned ephemeral one, is genuinely rejected by that same, untouched guard. This means the generic "bind `127.0.0.1:0`, use whatever port the OS gives you" pattern — while the right default choice, and the one this same codebase's own Phase 6 Performance Testing tests already use for tests that call the executor directly — cannot be applied here without either weakening the real SSRF logic (forbidden) or having these three tests stop exercising the real, SSRF-guarded HTTP endpoint (a real coverage loss, also forbidden). The fix instead makes the test binary itself start a real, minimal, self-contained target bound to that one specific, already-production-significant port — lazily, once per test process, shared safely across parallel test execution via `tokio::sync::OnceCell` — so no developer action is ever required and no port collision is possible.

Verified reproducibly: two independent full `cargo test` runs from a genuinely clean environment (confirmed via `netstat` immediately before each), both `298 passed, 0 failed, 2 ignored`. The three previously-failing tests individually confirmed passing, twice in isolation and twice as part of the full suite. No process or listening socket left behind after the test binary exits.

Final Verdict: FIXED. See §19.

---

2. Original Problem

`api_tests.rs` drives every one of its tests through the real, in-process `axum::Router` via `tower::ServiceExt::oneshot` — a deliberate, established convention (this file's own module doc comment: "these drive real HTTP requests through the real `axum::Router` in-process... rather than calling handler/store functions directly"). For the Phase 4 API Testing tests specifically, that real router dispatch ends in a genuine outbound `reqwest` call made by `executor::CompiledApiExecutor`/`execute_plan_once` against whatever URL the test supplied. Three of those tests supplied `HEALTH_URL = "http://127.0.0.1:8080/health"` and then asserted on the *result of a successful response* (`task_status == "PASSED"`, a specific response body, a specific real observed status) — assertions that can only be true if something is genuinely listening and responding on `127.0.0.1:8080` at test time. Nothing in the test process itself ever started such a listener; it relied entirely on a developer (or a previous manual `cargo run` session) having one already running. `cargo test` in a genuinely clean checkout, with nothing pre-started, therefore failed those 3 tests every time.

3. Baseline Audit

Performed before any change, per the brief's own required ordering. `git status` confirmed a clean, expected working tree matching the state left by the prior session. Ran the affected group with port 8080 confirmed free (`netstat -ano | grep ":8080"` -> no output):

```
running 8 tests
test api_tests::phase4_api_test_rejects_an_invalid_url_before_any_execution ... ok
test api_tests::phase4_api_test_rejects_a_disallowed_ssrf_target_via_the_existing_real_guard ... ok
test api_tests::phase4_api_test_rejects_an_unsupported_http_method ... ok
test api_tests::phase4_api_test_project_isolation_enforced ... ok
test api_tests::phase4_api_test_rbac_viewer_forbidden_operator_allowed ... ok
test api_tests::phase4_api_test_query_parameters_and_headers_genuinely_reach_the_real_request ... FAILED
test api_tests::phase4_api_test_assertion_mismatch_genuinely_fails_not_a_fake_pass ... FAILED
test api_tests::phase4_api_test_executes_a_real_request_and_passes_a_real_status_assertion ... FAILED

test result: FAILED. 5 passed; 3 failed
```

Exactly 3, confirmed by real output, not assumption. Every failure showed the identical real symptom: `CONNECTION_FAILED: error sending request for url (http://127.0.0.1:8080/health...)`.

Also searched, per the brief's required checklist:

- `127.0.0.1:8080`/`localhost:8080` across the whole crate (§4 has the full classification — most hits are unrelated: `execution_plan::DEFAULT_TARGET`, executor default-param strings, string-classification unit tests, none of which make a real network call).
- `#[ignore]` across the whole crate — found exactly 2, both pre-existing, both in `ai_generation.rs`/`intelligence.rs`, both already documented (`#[ignore = "requires a real, running local Ollama server..."]`), both genuinely unrelated to port 8080 and out of this fix's scope.
- Any existing self-contained local test-server helper — found one, in `performance_testing.rs`'s own test module (`spawn_test_target`, binds `127.0.0.1:0`) — inspected in full (§9 explains why it could not be reused as-is here).

4. Root Cause

Test	File	External Dependency	Why
`phase4_api_test_executes_a_real_request_and_passes_a_real_status_assertion`	`api_tests.rs:3178`	`HEALTH_URL` must genuinely respond `200 "ok"`	Asserts `task_status == "PASSED"`, `result.status == 200`, `response_body_preview == "ok"`, and a real, present `request_duration_ms`
`phase4_api_test_assertion_mismatch_genuinely_fails_not_a_fake_pass`	`api_tests.rs:3214`	`HEALTH_URL` must genuinely respond `200` (so a configured `404` assertion genuinely mismatches, producing `FAILED`, not `INFRA_ERROR`)	Asserts `task_status == "FAILED"` and `assertions[0].actual == "200 OK"` — both require a real successful response to compare against
`phase4_api_test_query_parameters_and_headers_genuinely_reach_the_real_request`	`api_tests.rs:3298`	`HEALTH_URL` must genuinely respond `200`	Asserts `task_status == "PASSED"` after sending real query params/headers alongside the request

Every other test in the file that also references `HEALTH_URL` (`phase4_api_test_rejects_an_unsupported_http_method`, `phase4_api_test_rbac_viewer_forbidden_operator_allowed`, `phase4_api_test_project_isolation_enforced`) only needs the URL to be *SSRF-valid* (it never reaches the network at all, or the assertion is only on the outer HTTP `200`/`403`/`400` status of the `/api-testing/execute` endpoint itself, which is independent of whether the inner request succeeded) — confirmed by direct code reading (§3) and by their already passing in the clean-environment baseline run above.

5. Files Inspected

`api_tests.rs` (in full, and specifically every `HEALTH_URL` reference), `executor.rs` (`is_blocked_ip`/`ORCHESTRATOR_PORT`/`validate_target_url`), `execution_plan.rs` (`DEFAULT_TARGET`), `performance_testing.rs` (`spawn_test_target`, the existing self-contained local-target pattern), `api.rs` (the real `/health` handler's own exact response body), `Cargo.toml` (confirming `tokio` is already a dependency with the features this fix needs, so no new dependency was required).

6. Files Changed

File	Change	Why
`services/orchestrator/src/api_tests.rs`	Added a `static HEALTH_TARGET_READY: tokio::sync::OnceCell<()>` and an `async fn ensure_local_health_target()`; added one `ensure_local_health_target().await;` call at the top of each of the 3 affected tests	The actual self-contained fixture, and its wiring into exactly the tests that need it

No other file was touched. `executor.rs`'s real SSRF guard, `execution_plan.rs`'s `DEFAULT_TARGET`, and every production HTTP-handling code path are byte-for-byte unchanged.

7. Previous Test Architecture

```
cargo test
   v
api_tests.rs's 3 affected tests dispatch a real HTTP request to
http://127.0.0.1:8080/health via the real router -> real CompiledApiExecutor
   v
expects 127.0.0.1:8080 to already have a real listener
   v
fails with CONNECTION_FAILED if no developer manually started one
```

8. New Test Architecture

```
cargo test
   v
first of the 3 affected tests to run calls ensure_local_health_target()
   v
tokio::sync::OnceCell::get_or_init — only the first caller actually runs
the init closure; every concurrent/later caller awaits the same result
   v
binds 127.0.0.1:8080 (the one, real, SSRF-allowed loopback port —
not an ephemeral one; see §9/§11 for why)
   v
   +-- bind succeeds -> spawns a real, minimal axum server serving /health
   +-- bind fails with AddrInUse -> a real server (e.g. a developer's own
   |   manually-started dev orchestrator) already owns it; reused as-is,
   |   since its /health route returns the identical real "ok" body
   +-- any other bind error -> the test panics with a clear, real message
   v
real AIQA executor performs the exact same real reqwest call as before
   v
real assertions against the real response
   v
no explicit shutdown needed — the fixture is an in-process tokio task in
the same process as the test binary; the OS reclaims the port the moment
that process exits, exactly like every other tokio::spawn'd task already
in this codebase's own tests
```

9. Test Server Design

- **Framework:** `axum` — already a dependency, already the exact framework the real orchestrator itself and `performance_testing.rs`'s own `spawn_test_target` use for self-contained test targets. No new dependency was added.
- **Routes:** exactly one — `GET /health`, returning the literal `"ok"` string via `axum::routing::get(|| async { "ok" })`. This byte-for-byte matches the real orchestrator's own production `health()` handler (`api.rs:209-211`: `async fn health() -> &'static str { "ok" }`), so `response_body_preview == "ok"` and every other assertion these tests make holds true whether the response came from this fixture or a real, manually-running dev instance.
- **Port:** the literal `8080` — **not** an OS-assigned ephemeral port. This is the one deliberate, explained departure from the brief's own default guidance (§11 gives the full reasoning): the production SSRF guard these tests must genuinely exercise (`executor::is_blocked_ip`) only allows loopback on this exact, fixed port; an ephemeral port would be rejected by that same, unmodified, correct security check on every single attempt, making the tests permanently fail rather than pass. This is not a "replacement" hardcoded port in the sense the brief warns against (introducing a new, arbitrary fixed value) — it is the *same*, pre-existing, already-production-significant port these tests' own `HEALTH_URL` constant already named, and the same one `execution_plan::DEFAULT_TARGET` and this file's own pre-existing `phase4_api_test_rejects_a_disallowed_ssrf_target_via_the_existing_real_guard` test (unmodified by this fix) already treat as the canonical, allowed exception.
- **Ephemeral-port mechanism reconsidered and rejected for this specific case:** `performance_testing.rs`'s own `spawn_test_target()` (`127.0.0.1:0`) was fully audited (§3, §5) as the "existing self-contained local test-server helper" the brief asked to check first. It works precisely *because* its callers (`run_workload`/`execute_plan_once` called directly) never pass through `execution_plan::validate_plan`'s SSRF check at all — the check runs once, earlier, only in the real HTTP-endpoint code path these three `api_tests.rs` tests specifically need to exercise. Reusing that exact helper here was not possible without either bypassing the very code path under test (a real coverage loss) or weakening the guard (forbidden). This reasoning is recorded here rather than silently choosing a workaround.
- **Lifecycle/discovery:** `tokio::sync::OnceCell::get_or_init` — safe under `cargo test`'s default parallel execution (multiple tests, run on separate threads within the same process, can call this concurrently; only one actually performs the bind, every other caller correctly awaits the same result). No `--test-threads=1` requirement, no manual coordination.
- **Cleanup:** none needed, and none faked. The listener/server task is an in-process `tokio::spawn`, alive only as long as the test binary's own process is; there is no separate OS process to reap, and the OS releases the socket automatically the instant the test binary exits — the same, standard behavior every other `tokio::spawn`'d test fixture in this codebase already relies on (e.g. `performance_testing.rs`'s own `spawn_test_target`).

10. Before vs After Coverage

Test	Before Behavior	After Behavior	Coverage Preserved?
`phase4_api_test_executes_a_real_request_and_passes_a_real_status_assertion`	Real AIQA executor -> manually-running external orchestrator's `/health` -> real 200 "ok" -> assert PASSED	Real AIQA executor -> test-started, self-contained `/health` fixture -> real 200 "ok" -> assert PASSED	Yes — identical real HTTP call, identical real assertions
`phase4_api_test_assertion_mismatch_genuinely_fails_not_a_fake_pass`	Real AIQA executor -> manual server -> real 200 -> real mismatch vs configured 404 -> assert FAILED	Real AIQA executor -> self-contained fixture -> real 200 -> real mismatch vs configured 404 -> assert FAILED	Yes
`phase4_api_test_query_parameters_and_headers_genuinely_reach_the_real_request`	Real AIQA executor with real query params/headers -> manual server -> real 200 -> assert PASSED	Real AIQA executor with real query params/headers -> self-contained fixture -> real 200 -> assert PASSED	Yes

The only thing removed, in every case, is the manual external dependency. No assertion was loosened, removed, or replaced with a mock. `CompiledApiExecutor`/`execute_plan_once` still perform a genuine `reqwest` HTTP request in every case — never stubbed, never short-circuited.

11. Security Review

- **The production SSRF guard (`executor::is_blocked_ip`/`ORCHESTRATOR_PORT`/`validate_target_url`) was not modified at all** — confirmed by `git diff` showing zero changes to `executor.rs` in this fix (the only file touched is `api_tests.rs`).
- **No production bypass was added.** The fixture doesn't touch validation logic; it makes the *target* real, it does not relax what targets are considered valid.
- **The fixture is compiled only as part of the test binary** (`api_tests.rs` is `#[cfg(test)] mod api_tests;` at the crate root) — it does not exist in, and cannot be reached from, a production build.
- **Why binding the real, production-significant port 8080 from a test is safe, not a weakening:** this is the exact address production code already treats as "the orchestrator's own, self-referential address" (`execution_plan::DEFAULT_TARGET`) — a real, intentional exception in the security model, not a new one this fix introduces. The fixture only ever serves a static `"ok"` string on `/health`; it cannot be used to reach any other resource, has no state, and performs no authentication bypass of any kind.
- **The `AddrInUse` fallback does not weaken anything either** — it means "someone else already owns this specific, already-allowed address," and defers to whatever real service is already there (most plausibly a real, manually-started dev orchestrator instance, whose own `/health` route is the exact same production code these tests are implicitly also exercising).

12. Test Results

Exact commands, exact results, no manually-started server at any point (confirmed via `netstat` immediately before each run):

```
$ netstat -ano | grep ":8080"          # -> no output, confirmed clean
$ cargo test --lib phase4_api
running 8 tests
test api_tests::phase4_api_test_project_isolation_enforced ... ok
test api_tests::phase4_api_test_rejects_an_unsupported_http_method ... ok
test api_tests::phase4_api_test_rejects_an_invalid_url_before_any_execution ... ok
test api_tests::phase4_api_test_rejects_a_disallowed_ssrf_target_via_the_existing_real_guard ... ok
test api_tests::phase4_api_test_assertion_mismatch_genuinely_fails_not_a_fake_pass ... ok
test api_tests::phase4_api_test_executes_a_real_request_and_passes_a_real_status_assertion ... ok
test api_tests::phase4_api_test_query_parameters_and_headers_genuinely_reach_the_real_request ... ok
test api_tests::phase4_api_test_rbac_viewer_forbidden_operator_allowed ... ok

test result: ok. 8 passed; 0 failed; 0 ignored; 0 measured; 292 filtered out; finished in 0.26s
```

All 3 previously-failing tests: **PASS**, with the other 5 in the same group unaffected.

13. Repeatability Results

The affected group was run three times in independent `cargo test` invocations (each a fresh process):

Run	Result	Notes
1	8 passed, 0 failed (0.26s)	First bind of the fixture
2	8 passed, 0 failed (0.24s)	Fresh process, fresh bind, no leaked state from run 1
3	8 passed, 0 failed (0.30s)	Same, plus explicit process/port-leak check below

After run 3:

```
$ tasklist | grep -i aiqa            # -> no leaked aiqa processes
$ netstat -ano | grep "LISTENING.*8080"   # -> port 8080 not held by any LISTENING socket
```

No leaked test-binary process, no leaked listening socket, confirming the fixture's lifecycle is genuinely scoped to each test process and cleans up naturally.

14. Full Backend Result

This is the critical section. Two independent full `cargo test` runs, both from a genuinely clean environment (`netstat` confirmed no `LISTENING` socket on 8080 before either run):

```
Run 1: test result: ok. 298 passed; 0 failed; 2 ignored; 0 measured; 0 filtered out; finished in 8.23s
Run 2: test result: ok. 298 passed; 0 failed; 2 ignored; 0 measured; 0 filtered out; finished in 8.18s
```

298 passed, 0 failed, 2 ignored — both runs. The 2 ignored are the pre-existing, legitimately-documented, unrelated Ollama-dependent tests (§3); no new `#[ignore]` was added anywhere. `298` is exactly `295` (the previous no-server baseline) `+ 3` (the newly-fixed tests) — arithmetic that confirms precisely the 3 targeted tests changed outcome and nothing else did.

15. Regression Results

#	Check	Result
1	Exact affected tests (`phase4_api_test_executes_a_real_request_and_passes_a_real_status_assertion`, `phase4_api_test_assertion_mismatch_genuinely_fails_not_a_fake_pass`, `phase4_api_test_query_parameters_and_headers_genuinely_reach_the_real_request`)	All 3 pass, no manual server
2	Affected integration test file/group (`phase4_api*`, 8 tests)	8/8 passed
3	Repeat affected tests	3 consecutive runs, 8/8 each (§13)
4	Full backend, no manual server	298/298, 0 failed, 2 ignored — twice (§14)
5	Backend check	`cargo check --lib --tests --bins` — clean
6	Relevant Performance Testing tests	`performance_testing::tests::*` — 30/30 passed, unaffected
7	Existing API executor tests (Phase 6 ERROR-fix group, which also touches `HEALTH_URL`)	`phase6_error_*` — 5/5 passed, unaffected
8	Frontend relevant tests	Not applicable — no shared contract or frontend file was changed by this fix
9	Full frontend suite	Not applicable — no frontend file was touched (confirmed via `git status`)
10	Typecheck/build (frontend)	Not applicable, same reason

16. Fake/Stub Audit

Full keyword sweep of the actual diff (`git diff src/api_tests.rs`, this session's real changes) for `fake|mock|demo|sample|stub|dummy|placeholder|hardcod|127.0.0.1:8080|localhost:8080|TODO|FIXME|ignore|skip|setTimeout|sleep|test-only bypass`. Every hit reviewed:

- References to `127.0.0.1:8080` — all are either the *existing*, unmodified `HEALTH_URL` constant, or this fix's own doc comments *explaining* why that specific, pre-existing port is used (not a new hardcode).
- `fake_pass` — part of a pre-existing test *name* (`..._genuinely_fails_not_a_fake_pass`), not new fake logic.
- `sample`/`RequestSample`/`real_sample` — all from the *previous* (unrelated) Phase 6 ERROR-fix session's own tests in this same file, not new fake data.
- One `sleep` — `tokio::time::sleep(...)` inside the pre-existing `phase6_error_normal_completion_regression_still_reaches_completed_not_error` test's real polling loop (waiting for a real, in-flight async operation to finish), not a timing hack standing in for real behavior; unrelated to and unmodified by this fix.
- No `mock`, `demo`, `stub`, `dummy`, `placeholder`, `TODO`, `FIXME`, `ignore`, `skip`, or `setTimeout` hits at all in this fix's own changes.

Specifically proven:

1. **No manual server is required** — §12/§14, verified with `netstat` confirming a clean environment before every run.
2. **No fixed port is required beyond the one already production-significant, pre-existing port** — §9 explains precisely why an ephemeral port cannot be substituted here without weakening real SSRF logic.
3. **No replacement hardcoded port exists** — the fixture binds the exact same `8080` the tests' own pre-existing `HEALTH_URL` already named; nothing new was invented.
4. **Real AIQA executor logic is still exercised** — `CompiledApiExecutor`/`execute_plan_once` make a genuine `reqwest` call in every affected test, unchanged (§10).
5. **No production SSRF bypass was introduced** — `executor.rs` has zero diff in this fix (§11).
6. **No tests were newly ignored/skipped** — zero new `#[ignore]` anywhere; the pre-existing 2 are untouched and were already legitimately documented.
7. **No assertions were weakened** — every original assertion in the 3 affected tests is byte-for-byte unchanged; only a fixture-startup call was added.
8. **Temporary server cleanup exists** — no explicit cleanup code is needed or was added; the in-process task's lifetime is bound to the test process itself, verified via the process/port-leak check in §13.
9. **Tests can run repeatedly** — §13, three consecutive runs, all green.
10. `cargo test` **works from a clean environment** — §14, twice, from a verified-clean starting state.

17. Known Limitations

- **The fixture binds a fixed, non-ephemeral port (`8080`) rather than following the generic ephemeral-port pattern this brief otherwise prefers.** This is a deliberate, fully-reasoned exception (§9/§11), not an oversight: it is the one port the *unmodified, production* SSRF guard genuinely allows for a loopback target, and these three tests specifically need to exercise that real, SSRF-guarded HTTP endpoint end-to-end. An ephemeral port would make every one of them fail the SSRF check, every time, by design.

- **If two separate `cargo test` invocations for this same crate were somehow run fully concurrently against the same machine** (e.g., two parallel CI shards on one host, outside the normal single `cargo test` process model this fix targets), both would attempt to bind `8080` in their own respective processes; the second would hit the real `AddrInUse` fallback and simply reuse the first's fixture instead of erroring — a real, working degradation path, not a crash, but worth noting as the one scenario where "port collision" in the traditional sense could occur, and it is handled gracefully rather than ignored.

- **The real orchestrator dev server and this test fixture cannot both be started manually and by `cargo test` on the same port simultaneously without one yielding to the other** — by design, this is intentional and safe (§8/§11): whichever gets there first serves the identical real `/health` response either way.

18. Evidence Matrix

Requirement	Code Evidence	Test Evidence	Verdict
No manual server	`ensure_local_health_target`, `api_tests.rs`	§12/§14, `netstat`-confirmed clean runs	■
No fixed *replacement* port	Reuses the existing `HEALTH_URL`/`8080`, no new constant	§9, §16.3	■
Ephemeral port pattern considered	§9's `spawn_test_target` audit	N/A — genuinely inapplicable here, explained	■■ (documented, justified exception)
Real executor	`CompiledApiExecutor::execute`/`execute_plan_once`, unchanged	§10 before/after table	■
Existing coverage preserved	Zero assertions changed	§10	■
Server cleanup	In-process `tokio::spawn`, no separate OS process	§13 process/port check	■
Repeatable tests	`OnceCell`, no per-test rebind	§13, 3 runs	■
Parallel-safe	`tokio::sync::OnceCell::get_or_init`	§12/§14, default parallel `cargo test`	■
SSRF not weakened	Zero diff in `executor.rs`	§11	■
No newly ignored tests	Zero new `#[ignore]`	§16.6	■
Full `cargo test` clean	298/298, 2 pre-existing ignored	§14, twice	■

19. Final Verdict

FIXED.

1. `cargo test` runs without manually starting a server — yes (§12, §14, verified with `netstat`).
2. Previously failing tests now pass — yes, all 3, individually and as part of the full suite (§12, §14).
3. No fixed port dependency remains that a developer must manually satisfy — yes; the port is now self-provisioned by the test process itself.
4. Tests use dynamically assigned ports *where that pattern is actually applicable* — the existing `performance_testing.rs` self-contained tests already do (unaffected, §15 row 6); for these 3 specific tests, a fixed, already-production-significant port was the correct and only real-security-preserving choice, fully justified in §9/§11/§17.
5. Real AIQA executor logic is still tested — yes, unchanged (§10).
6. No production security bypass was introduced — yes, confirmed zero diff to `executor.rs` (§11).
7. No tests were hidden/skipped to get green — yes, confirmed by the fake/stub audit (§16).
8. Temporary server lifecycle is clean — yes, verified via process/port-leak checks (§13).
9. Affected tests pass repeatedly — yes, 3 consecutive runs (§13).
10. Full backend suite has 0 unexpected failures — yes, 298/298 twice, only the 2 pre-existing, legitimately-documented, unrelated ignores remain (§14).

20. Final Questions

1. **Which exact tests previously depended on port 8080?** `phase4_api_test_executes_a_real_request_and_passes_a_real_status_assertion`, `phase4_api_test_assertion_mismatch_genuinely_fails_not_a_fake_pass`, `phase4_api_test_query_parameters_and_headers_genuinely_reach_the_real_request` — confirmed by actually running the group with no server up (§3).

2. **Why did each one need an external server?** Each asserts on the *outcome of a genuinely successful* real HTTP response (a specific `task_status`, a specific response body, a specific observed status) — see §4's table for the exact assertion in each case.

3. **Did the tests actually need AIQA itself running, or only a target HTTP server?** Only a target HTTP server for the outbound call to succeed — the AIQA backend under test is the same in-process `AppState`/router every other test in this file already uses via `tower::ServiceExt::oneshot`; no second AIQA instance was ever needed.

4. **What self-contained server architecture was chosen?** A single-route (`GET /health`) `axum::Router`, bound once per test-binary process via `tokio::sync::OnceCell`, spawned as an in-process `tokio` task (§8, §9).

5. **Is any fixed port still used?** Yes, deliberately — `8080`, the one port the unmodified, production SSRF guard genuinely allows for a loopback target; not a new/replacement port, the same one the tests' own pre-existing `HEALTH_URL` constant already named (§9, §11).

6. **How is the port assigned?** It is not dynamically assigned for these 3 tests — bound literally, for the reason in §9/§11. The codebase's other self-contained test fixture (`performance_testing.rs`'s `spawn_test_target`) does use a real, OS-assigned ephemeral port (`127.0.0.1:0`) where that pattern is actually applicable (its callers bypass the SSRF-guarded HTTP layer entirely) — unaffected, unchanged, still in use (§15 row 6).

7. **How does each test discover the actual URL?** It doesn't need to discover anything — `HEALTH_URL` was already a compile-time constant naming the one allowed address; `ensure_local_health_target()` simply guarantees something real is listening there before the test proceeds.

8. **Is the real AIQA executor still performing HTTP requests?** Yes, unchanged, genuine `reqwest` calls via `CompiledApiExecutor`/`execute_plan_once` (§10).

9. **How is the server cleaned up?** It isn't explicitly torn down — it is an in-process task scoped to the test binary's own process lifetime; the OS reclaims the port the instant that process exits (§9, §13).

10. **Can tests run twice consecutively?** Yes — verified 3 times for the affected group (§13) and twice for the full suite (§14), all green, no leaked process or port (§13).

11. **Can `cargo test` run from a clean environment?** Yes — verified with `netstat` confirming no listener on 8080 immediately before every run in §12/§14.

12. **Were any tests ignored/skipped?** No new ones — the only `#[ignore]`s in the crate are the 2 pre-existing, legitimately-documented, Ollama-dependent ones, unrelated to and untouched by this fix (§3, §16.6).

13. **Was production SSRF protection changed?** No — zero diff to `executor.rs` in this fix, confirmed directly (§11).

14. **What exact full backend result was obtained with no manually running server?** `298 passed; 0 failed; 2 ignored`, reproduced twice (§14).

15. **Does any limitation remain?** The fixed-port choice for these 3 specific tests is a deliberate, explained departure from the generic ephemeral-port default, required by the real, unweakened SSRF guard's own design — not a limitation this fix could have avoided without either bypassing the code path under test or weakening production security (§17).

End of report.